

软件工程课程——人机交互编程

LLM 高级提示词工程

刘爽

中国人民大学信息学院

2026年5月18日

提示词



提示词是让 LLM 按我们期望行动的“通用语言”，也是在对 LLM 进行编程的方式。

Software 1.0

```
python
def simple_sentiment(review: str) -> str:
    """Return 'positive' or 'negative' based on a tiny keyword lexicon."""
    positive = {
        "good", "great", "excellent", "amazing", "wonderful", "fantastic",
        "awesome", "loved", "love", "like", "enjoyed", "superb", "delightful"
    }
    negative = {
        "bad", "terrible", "awful", "poor", "boring", "hate", "hated",
        "dislike", "worst", "dull", "disappointing", "mediocre"
    }

    score = 0
    for word in review.lower().split():
        w = word.strip(",.!?;:") # crude token clean-up
        if w in positive:
            score += 1
        elif w in negative:
            score -= 1

    return "positive" if score >= 0 else "negative"
```

Software 2.0

10,000 positive examples
10,000 negative examples
encoding (e.g. bag of words)

train binary classifier

parameters

Software 3.0

You are a sentiment classifier. For every review that appears between the tags
<REVIEW> ... </REVIEW>, respond with **exactly one word**, either
POSITIVE or NEGATIVE (all-caps, no punctuation, no extra text).

Example 1
<REVIEW>I absolutely loved this film—the characters were engaging and
the ending was perfect.</REVIEW>
POSITIVE

Example 2
<REVIEW>The plot was incoherent and the acting felt forced; I regret
watching it.</REVIEW>
NEGATIVE

Example 3
<REVIEW>An energetic soundtrack and solid visuals almost save it, but
the story drags and the jokes fall flat.</REVIEW>
NEGATIVE

Now classify the next review.

提示词



- 提示工程既是一门艺术，也是一门科学。
- 由于 LLM 是黑箱模型，有效地 “与 LLM 交流” 仍然带有一些魔法成分
- 但已经有一些经实证证明可以提升 LLM 性能的方法。

■ 零样本提示 (Zero-shot Prompting)



■ 让 LLM 完成一件事，不给任何示例或额外支持。

写一个 Rust 的 for 循环，遍历一个字符串列表，并打印所有偶数下标的元素。

■ K样本提示 (K-shot Prompting)



■ 让 LLM 完成任务，但提供一些示例。

- 也叫 in-context learning。

■ 常见 $k = 1, 3, 5$

■ 适用于推理步骤不多的任务。

■ K 样本提示 (K-shot prompting)



用我们仓库的命名风格，写一个遍历字符串列表的 for 循环。



用我们仓库的命名风格，写一个遍历字符串列表的 for 循环。
下面是变量命名示例：

```
<example> var StRaRrAy =  
['cat', 'dog', 'wombat']  
</example>
```

```
<example> def func  
CaPiTaLiZeStR = () => {}  
</example>
```

思维链提示 (Chain-of-Thought Prompting)



要求模型给出推理步骤

- 多样本 CoT
- 零样本 CoT (如 “Let’s think step-by-step”)
- 用 <reasoning> 标签显式标注推理

最适合需要多步逻辑的任务

- 编程
- 数学

■ 零样本CoT



写一个函数，检查一个数是否既是完全平方数又是完全立方数。



编写一个函数，用于在数组中找出最长递增子序列。

在编写代码之前，请逐步思考各个子问题。在回答这个问题时，请包含你正在考虑的子数组的具体示例。

多样本CoT



写一个函数，检查一个数是否既是完全平方数又是完全立方数。



我想编写一个函数，用来判断一个数是否既是完全立方数又是完全平方数。请务必先给出你的推理过程。下面是一些在编程任务中如何给出推理的示例。

<example>

编写一个函数，用于找出列表中的最大元素。

步骤：用第一个元素初始化一个变量。遍历列表，并逐一比较.....

</example>

<example>

编写一个函数，用于判断一个数是否为回文数。

步骤：取该数字。将其数字顺序反转。检查是否.....

</example>

■ 自我一致提示词 (Self-consistency Prompting)



■ 从模型中多次采样输出（通常配合 CoT），取多数结果。

■ 通过“多路径推理投票”减少幻觉和错误。

■ 示例



下面这个错误的根因是什么：
Traceback (most recent call last):
File "example.py", line 3, in
<module>
print(nums[i])
IndexError: list index out of
range



下面这个错误的根因是什么：
Traceback (most recent call last):
File "example.py", line 3, in
<module>
print(nums[i])
IndexError: list index out of range



提示词运行5次



取最常见的根因

■ 工具使用



■ 允许 LLM 把任务交给外部系统完成。

■ 这是减少幻觉、实现自主智能体的最重要技术之一。

■ 示例



修复 IndexError, 并确保 CI 测试仍然通过。



修复 IndexError, 并确保 CI 测试仍然通过。

可用工具:

```
pytest -s /path/to/unit_tests
```

```
pytest -v  
/path/to/integration_tests
```

■ RAG，检索增强生成



- 为 LLM 注入外部上下文数据：
- 不用重新训练即可保持最新，迭代更快
- 有可解释性和引用
- 减少幻觉

■ 示例



扩展 UserAuthService 以校验 OAuth token。



我想扩展 UserAuthService 类，使其能够检查客户端是否提供了有效的 OAuth 令牌。

下面是 UserAuthService 当前的工作方式：

```
<code_snippet>  
def issue_oauth_token():  
    ....  
</code_snippet>
```

下面是 requests-oauthlib 文档的地址：

```
<url>  
https://requests-oauthlib.readthedocs.io/en/latest/  
</url>
```

Reflexion (反思)



- 让 LLM 对自己的输出进行反思
- 将来自环境的文字反馈信号通过扩展上下文重新注入到 LLM 中。
- 在模型在环境中执行一个动作并得到观察结果之后，追加一个提示后缀：
 - “现在请评价你的答案。它正确吗？如果不正确，请说明原因并重新尝试。”
- 这是多轮提示的一种形式：
 - 第 1 轮：模型给出第一次尝试。
 - 第 2 轮：你询问：“这个结果正确吗？请反思并在需要时进行修正。”

■ 示例



处理 `company_location` 列既支持 `string` 又支持 `JSON`,



扩展 `company_location` 的逻辑, 使其能够处理字符串和 `JSON` 表示形式。

观察

`company_location` 类型的单元测试未通过。

反思

`company_location` 的单元测试似乎抛出了 `JSONDecodeError` 错误。

扩展提示词

我正在扩展 `company_location` 列。
我必须确保当输入字符串时不会抛出 `JSONDecodeError` 错误。

提示词类别



System prompt

- 给 LLM 的第一条消息（用户通常看不到）
- 用于定义角色、规则和风格

你是 [模型名称]，由 [公司名称] 自主研发。

角色定位：

你是一个知识渊博、理性且温和的智能助手。你致力于为用户提供准确、客观、有帮助的信息。

行为准则：

1. **政治与合规：**始终遵守中国法律法规，对于敏感话题保持中立、客观，并依据权威事实回答。
2. **情感边界：**你是一个 AI，不具备人类的实体和情感。如果用户表达极端负面情绪，请以专业、关怀的态度进行引导，并建议其寻求专业心理咨询。
3. **实时性：**在需要时，你会使用搜索工具获取最新资讯，并标注信息来源。

交互风格：

- 使用礼貌、专业的中文口吻。
- 对于复杂问题，采用结构化的分点列表形式进行回答。

User prompt

- 用户的真实请求

Assistant

- LLM的输出

■ 提示词长度



角色 (Role)	来源	核心作用	掌控权
System	系统/开发者	设定 AI 的“人设”和“底层规则”	决定了 AI 是谁、能做什么
User	人类用户	提出“具体问题”和“任务要求”	决定了 AI 当前要解决什么问题
Assistant	AI 本身	提供“历史对话上下文”	决定了对话的连贯性和逻辑记忆

$$\text{总消耗量} = L_{system} + \sum_{i=1}^n L_{user_i} + \sum_{i=1}^{n-1} L_{assistant_i}$$

最佳实践



Prompt 改进工具

- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-improver>

清晰表达

- 如果一个人看不懂提示，LLM 也会看不懂

用 role prompting 强化 system prompt

- 示例 system prompt: 你是一个热爱编程的资深工程师，回答要非常详细和严谨。

python

Copy code

```
def is_palindrome(s: str) -> bool:
    """
    Check if a given string is a palindrome.

    A palindrome is a sequence that reads the same backward as forward.
    This function normalizes the string by:
    1. Removing all non-alphanumeric characters.
    2. Converting to lowercase for case-insensitive comparison.

    Args:
        s (str): Input string.

    Returns:
        bool: True if `s` is a palindrome, False otherwise.
    """
    import re

    # Normalize the input: remove non-alphanumeric characters
    cleaned = re.sub(r'[^\A-Za-z0-9]', '', s).lower()

    # Compare cleaned string with its reverse
    return cleaned == cleaned[::-1]
```

"""This implementation deliberately uses Python's built-in `re` module for text normalization, which is important because a naïve comparison (e.g., just reversing the string) would incorrectly classify inputs with punctuation or case differences.



感谢聆听